

Sistemas Basados en Microprocesador

Integración y desarrollo de
una aplicación:
Controlador de una válvula termostática

Alumno:
Blanca Loma Mecha

Puesto Nº: 6

2025-26

Índice del documento

1	OBJETIVOS DE LA PRÁCTICA	2
1.1	Resumen de los objetivos de la práctica realizada	3
1.2	Acrónimos utilizados	3
1.3	Tiempo empleado en la realización de la práctica.	4
1.4	Bibliografía utilizada	4
1.5	Autoevaluación.....	5
2	RECURSOS UTILIZADOS DEL MICROCONTROLADOR	6
2.1	Diagrama de bloques hardware del sistema.....	7
2.2	Cálculos realizados y justificación de la solución adoptada.	7
	SOFTWARE.....	10
2.3	Descripción de cada uno de los módulos del sistema.....	11
2.4	Descripción global del funcionamiento de la aplicación. Descripción del autómata con el comportamiento del software (si procede).....	12
2.5	Descripción de las rutinas más significativas que ha implementado.	14
3	DEPURACION Y TEST	15
3.1	Pruebas realizadas.	16

1 OBJETIVOS DE LA PRÁCTICA

1.1 Resumen de los objetivos de la práctica realizada

1. Desarrollar una aplicación basada en RTOS, utilizando la tarjeta NUCLEO STM32F429ZI a partir de las especificaciones generales.
2. Conocer los elementos hardware y software abordados en la asignatura, y demostrar la capacidad de integrar estos conceptos en el desarrollo de la aplicación.
3. Analizar el funcionamiento y utilizar de forma autónoma algunos elementos no estudiados, e incluirlos en la aplicación propuesta.
4. Dividir las tareas del sistema en hilos de ejecución independientes.
5. Controlar la comunicación entre hilos mediante el uso de colas de mensajes y flags.
6. Verificar los proyectos individuales mediante tests específicos, previos a la integración de cada módulo.
7. Manejar temporizadores hardware para gestionar la temporización y sincronización de una aplicación.
8. Aprender a manejar cualquier periférico de mediana complejidad de un microcontrolador a partir de la documentación proporcionada por el fabricante.
9. Integrar el sensor VEML7700 y el LED de alta luminosidad, de forma autónoma, buscando y utilizando la documentación necesaria.
10. Implementar la comunicación I2C entre la tarjeta NUCLEO y el sensor VEML7700, haciendo uso de sus puertos de datos y reloj (SDA y SCL).
11. Generar una señal PWM para el LED de alta luminosidad, permitiendo modificar su ciclo de trabajo.
12. Automatizar el ajuste del LED de alta luminosidad, según la luminosidad que queramos que el sensor capte.
13. Controlar la pantalla LCD, mediante el protocolo SPI, para mostrar dos filas principales de información y dos barras ajustables.
14. Programar el joystick mediante interrupciones EXTI, aplicando filtrado de rebotes.
15. Configurar el módulo de comunicación con el PC, mediante la utilización del periférico USART, permitiendo recibir comandos en ASCII, y responder automáticamente a estos mediante la aplicación TeraTerm.
16. Reutilización de los proyectos realizados durante la evaluación ordinaria del bloque 3 y a lo largo de la asignatura.
17. Diseño de un autómata de estados principal, que gobierne el comportamiento global del dispositivo.
18. Gestión del almacenamiento circular de medidas periódicas, mediante el uso de un buffer indexado en memoria, con capacidad de sobreescritura y borrado.
19. Adaptar los proyectos individuales al uso real del proyecto principal.

1.2 Acrónimos utilizados

Acrónimos utilizados en el enunciado del diseño y en la realización de la memoria del diseño.

EXTI	External Interrupt Event Controller
SPI	Serial Peripheral Interface
I2C	Inter-Integrated Circuit
SDA	Serial DATA
SCL	Serial CLock
LCD	Liquid-crystal display
PWM	Pulse Width Modulation
USART	Universal Synchronous/Asynchronous Receiver/Transmitter
PC	Personal Computer
LED	Light Emmiting Diode
RGB	Red Green Blue
USB	Universal Serial Bus
CMSIS	Cortex Microcontroller Software Interface Standard
RTOS	Real-Time Operative System
SOH	Start of Header
CMD	Command
LEN	Length
EOT	End Of Transmission
ASCII	American Standard Code for Information Interchange
SBM	Sistemas Basados en Microprocesador
UPM	Universidad Politécnica de Madrid
MCU	MicroController Unit

1.3 Tiempo empleado en la realización de la práctica.

En la realización de la práctica para la convocatoria ordinaria, estimamos mi compañera y yo, unas 30 horas de trabajo. Debido a que gran parte de esta práctica ha sido una modificación de la anterior, considero estas horas en el recuento.

En la modificación de los bloques individuales estimo 12 horas, debido a que los módulos RGB, POT, LCD, JOYSTICK y HORA tuvieron mínimos cambios (aunque todos fueron revisados y mejorados), pero otros módulos como SENSOR_LUMINOSIDAD, PWM y COM_PC, han sido muy modificados.

Al módulo principal le estimo 20 horas, ya que he implementado cada módulo poco a poco.

A la realización del informe le estimo 8 horas.

En total el tiempo empleado ha sido de unas 70 horas.



[Tiempo empleado para realizar la práctica]: El tiempo total empleado ha sido de 70 horas.

1.4 Bibliografía utilizada

- [RD1] UPM. Documentación docente y enunciados de prácticas de la asignatura SBM, Curso 2025/26.
- [RD2] STMicroelectronics. UM1974 User Manual STM32 Nucleo-144 boards. [En línea]. Disponible en Moodle de la asignatura.
- [RD3] STMicroelectronics. RM0090 Reference manual STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/449 ARM-based 32-bit MCUs. [En línea]. Disponible en Moodle de la asignatura.
- [RD4] STMicroelectronics. STM32F427xx STM32F429xx Datasheet. [En línea]. Disponible en Moodle de la asignatura.
- [RD5] Mbed-Application-Board. Esquemático. [En línea]. Disponible en Moodle de la asignatura.
- [RD6] Ka wah core. Especificaciones técnicas y hoja de producto: Diodos LED de alta luminosidad (5mm, transparente, luz blanca). [En línea]. Disponible en: <https://www.amazon.es/Ka-wah-core-LED-Diodos/dp/B0C77LDCG6?th=1>
- [RD7] TECNOULAB. Especificaciones técnicas y hoja de producto: Sensor de luz ambiental VEML7700, Placa de Sensor de medición de luz de 120k Lux. [En línea]. Disponible en: https://www.amazon.es/dp/B0CLY5ZCNL?ref=ppx_yo2ov_dt_b_fed_asin_title

1.5 Autoevaluación.

Considero que he superado con solvencia los objetivos de adquisición obligatoria:

-Integración del RTOS (RA263, RA735, RA737, RA971, RA907): he diseñado una arquitectura multihilo, con 8 hilos independientes, sincronizados mediante colas de mensaje y flags. Y he utilizado otras herramientas de RTOS como los timers. Para este desarrollo, partía del proyecto ordinario que realicé en grupo, junto a mi compañera María.

-Uso de protocolos estándar y lectura de documentación (RA730, RA733, RA736): he utilizado la documentación conveniente para integrar correctamente la mbed-app-board y la tarjeta núcleo, así como el sensor VEML7700 y el LED de alta luminosidad. También me he ayudado de esta documentación para manejar el funcionamiento de los periféricos.

-Comunicación entre sistemas (RA970): he implementado la comunicación entre el PC y la tarjeta NUCLEO, mediante el periférico USART.

-Manejo de instrumentación electrónica (RA968): he utilizado el osciloscopio del laboratorio y el analizador lógico entregado por los profesores, para resolver problemas que me han surgido a lo largo del curso.

Elaboración de informe (RA738): como se comprueba en este documento, soy capaz de redactar un informe del proyecto.

Durante el desarrollo del proyecto he enfrentado varios problemas, algunos de ellos los comento a continuación:

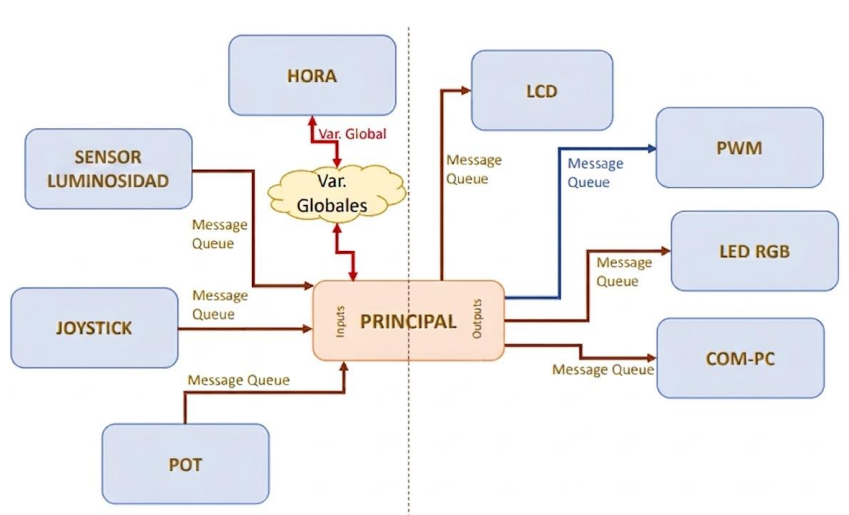
-Inicialmente conecté el LED de alta luminosidad a la protoboard donde tengo conectado el sensor VEML7700, pero la luminosidad que captaba el sensor no era adecuada. Tras varias comprobaciones entendí que el problema residía en la posición del led, ya que por muy cerca que lo pusiese del sensor, siempre captaba poca luminosidad. Finalmente dejé el LED colgando con dos cables macho-hembra, y al iluminar el sensor con la punta del led, sí que obtenía la luminosidad requerida.

-Tuve ciertos problemas también con el módulo de comunicación con el PC, debido a que el hilo funcionaba bien individualmente en sus test, pero en conjunto con el resto del código, tumbaba el proceso. Finalmente, mediante unos ajustes de código y la especificación de pila en este hilo, y algún otro, conseguí que funcionase correctamente.

-Finalmente, hallé ciertos problemas con mi ordenador personal, puesto que, al probar mi código en este, el LCD parpadeaba y a veces el programa se tumbaba antes de comenzar, dejando la inicialización a medias. Tras probar el código y mi placa con otros códigos y ordenadores, descubrí que el problema estaba en mi ordenador, y proseguí utilizando los ordenadores del laboratorio.

2 RECURSOS UTILIZADOS DEL MICROCONTROLADOR

2.1 Diagrama de bloques hardware del sistema.



2.2 Cálculos realizados y justificación de la solución adoptada.

Módulo RGB	Pin mbed-app-board	STM32F429	NUCLEO STM32F429ZI
R	DIP23	PD13	CN10.19 / D28
G	DIP24	PD12	CN10.21 / D29
B	DIP25	PD11	CN10.23 / D30

Valores programados: Los LEDs se han configurado con modo salida pull-push (GPIO_MODE_OUTPUT_PP), resistencia de pull-up (GPIO_PULLUP) y velocidad de respuesta muy alta (GPIO_SPEED_FREQ_VERY_HIGH).

Cálculos realizados: Usamos un timer periódico de 1ms, que incrementa una cuenta. Si el ciclo de trabajo seleccionado es mayor que la cuenta, pinta el LED. De esta forma, si se quiere el LED encendido constantemente, con un ciclo de 100, siempre accederá a pintar el LED.

Módulo JOYSTICK	Pin mbed-app-board	STM32F429	NUCLEO STM32F429ZI
Down	DIP12	PE12	CN10.26 / D39
Left	DIP13	PE14	CN10.28 / D38
Center	DIP14	PE15	CN10.30 / D37
Up	DIP15	PB10	CN10.32 / D36
Right	DIP16	PB11	CN10.34 / D35

Valores programados: Los pines del Joystick se han configurado como entradas asociadas a líneas de interrupción externa con detección de flanco de subida (GPIO_MODE_IT_RISING), resistencia de pull-down (GPIO_PULLDOWN) y velocidad de respuesta baja (GPIO_SPEED_FREQ_LOW). Adicionalmente, se ha habilitado y configurado el vector de interrupciones de la de la 15 a la 10.

Cálculos realizados: Usamos un timer periódico de 20ms, que incrementa una cuenta. Si llega a los 50 ticks, consideramos una pulsación larga (20ms*50ticks = 1s).

MÓDULO HORA:

Valores programados: Utilizamos variables globales de tipo integer.

Cálculos realizados: Usamos un timer periódico de 1 segundo, e incrementamos los segundos, ajustando con estos los minutos y horas.

Módulo POT	Pin mbed-app-board	STM32F429	NUCLEO STM32F429ZI
POT_1	DIP19	PA3	CN9.1 / A0
POT_2	DIP20	PC0	CN9.3 / A1

Valores programados: Los potenciómetros se han configurado con modo entrada analógica (GPIO_MODE_ANALOG) y sin resistencia (GPIO_NOPULL). El pin PA3 del potenciómetro 1, se asocia al canal 3 de ADC (ADC1_IN3) y el PC0 del potenciómetro 2, se asocia al canal 10 (ADC1_IN10). El periférico ADC1 se ha programado para trabajar en modo de conversión única, con alineación de datos a la derecha, resolución de 12 bits y tiempo de muestreo de 3 ciclos de reloj.

Cálculos realizados: Para obtener el voltaje de los potenciómetros usamos $voltaje = raw \cdot \frac{3.3}{4096}$, que representa el valor digital en bruto (raw), obtenido directamente del registro ADC, por el voltaje de referencia, entre la resolución para 12 bits.

Para convertir los voltios que recibimos de la función ADC_getVoltage, en la conversión para el potenciómetro 1 usamos $lux = 100 + voltios \cdot \frac{4000-100}{3.3}$, que representa el salto de 100 en 100, sumado a los voltios por el rango, entre el máximo voltaje. A este resultado, le aplicamos un redondeo simple, sumándole 0.5, para convertirlo a uint16_t. En la conversión para el potenciómetro 2 usamos $porcentaje = 5 + voltios \cdot \frac{95-5}{3.3}$, que representa el salto de 5 en 5, sumado a los voltios por el rango, entre el máximo voltaje. A este resultado, le aplicamos un redondeo simple, sumándole 0.5, para convertirlo a uint8_t.

Módulo LCD	Pin mbed-app-board	STM32F429	NUCLEO STM32F429ZI
MOSI	DIP5	PA7	CN7.14 / D11
SCK	DIP7	PA5	CN7.10 / D13
CS	DIP11	PD14	CN7.16 / D10
A0	DIP8	PF13	CN10.2 / D7
RESET	DIP6	PA6	CN7.12 / D12

Valores programados: Configuramos la interfaz SPI1 en modo maestro síncrono (ARM_SPI_MODE_MASTER), con polaridad y fase a 1 (ARM_SPI_CPOL1_CPHA1), con orden de bit más significativo primero (ARM_SPI_MSB_LSB), 8 bits para cara trama (ARM_SPI_DATA_BITS(8)) y frecuencia de reloj de 10 MHz. Se configuran los pines CS, A0 y RESET en modo salida push-pull (GPIO_MODE_OUTPUT_PP) sin resistencia de pull (GPIO_NOPULL).

Cálculos realizados: Se calcula que la página 0 usa los bytes 0-127, la página 1 del 128-255, la página 2 del 256-383 y la página 3 del 384-511. Para dibujar en las líneas 1 o 2 se usa $offset = 25 \cdot (symbol - '')$ como byte de inicio de cada carácter ASCII, restándole el carácter espacio y multiplicando por el tamaño de la matriz indexada de 25 bytes. A continuación, se utiliza un bucle iterativo de 12 columnas para realizar el direccionamiento de páginas dentro del buffer plano de 512 bytes. Primero se separan los bytes en par e impar para calcular su posición ($value1 = offset + i \cdot 2 + 1$ y $value2 = offset + i \cdot 2 + 2$). Luego dependiendo de la línea que sea, se almacenan los valores en la posición actual del buffer $buffer[i + posicionL1]$, o se le añade 128 bytes. Estando el valor 2 siempre a 128 bytes del valor 1. Se utiliza una prevención de desbordamiento al realizar $if((i + posicion) < 128)$.

Por último, para dibujar las barras de luminosidad usamos las columnas 110 y 120. Para la altura de los pixeles, sabiendo que hay 32 pixeles en el LCD, usamos $altura = \frac{nivel \cdot 32}{4000}$. Se realiza un bucle inverso desde la página 3 a la página 0, de forma que:

- Si quedan 8 o más pixeles por pintar $if(altura_{temporal} \geq 8)$, se rellena la página actual competa con 0xFF.
- Si quedan entre 0 y 8 pixeles por pintar $if(8 > altura_{temporal} > 0)$, se rellenan los pixeles superiores necesarios $0xFF \ll (8 - altura_{temporal})$.
- Si quedan 0 pixeles por pintar, se borra con 0x00.

MÓDULO COM-PC:

Valores programados: Utilizamos la USART3 configurada en modo asíncrono, con un formato de comunicación de 9600 baudios, 8 bits de datos, 1 bit de stop y sin paridad. Se han habilitado los hilos independientes de transmisión ThTX y recepción ThRX, con pilas de 1024 bytes.

Cálculos realizados: Para el empaquetado y desempaquetado de tramas se utilizan cálculos de longitud total, $longitud = Payload + SOH + CMD + LEN + EOT$, añadiendo en la trama de transmisión 2 bytes también para \r\n para la visualización en Teraterm. A la hora de leer estas tramas, se envía la longitud del payload, $msg RX.length = BufferRX[2] - 4$, restándole los 4 bytes de la estructura, a la longitud de la trama total. Por último, para la realización de las medidas del buffer circular se utiliza $indice\ escribir = (indice\ escribir + 1) \% MAX_MEDIDAS$, e igual con el de leer, que asegura la sobreescritura circular, sin desbordar los límites de medidas.

Módulo PWM	STM32F429	NUCLEO STM32F429ZI
LED-> cátodo	GND	CN7.8 / GND
LED-> ánodo	PE9	CN10.4 / D6

Valores programados: Usamos un timer de propósito general (TIM1), configurado en modo PWM1 (configura el flanco, primero inactivo y luego activo), con configuración en Output Compare Mode y con 2 kHz de frecuencia al 50% de ciclo de trabajo (contando con SysClock a 168 MHz: prescaler a 167, period a 499 y pulse a 250). Configuramos también el pin PE9 en modo AF1 y unido a CHANNEL 1, el cual se conecta físicamente con el LED de alta luminosidad.

Cálculos realizados: Para el valor del pulso, a partir del ciclo de trabajo pedido, usamos la ecuación $valor\ pulso = \frac{ciclo \cdot 500}{100}$, correspondiente al ciclo multiplicado por el periodo+1 (500), entre 100, ya que el ciclo es un porcentaje. De esta forma, para un periodo entre 0 y 499, el 5% del ciclo corresponde al periodo 25, y el 95% al periodo 475.

Módulo SENSOR	STM32F429	NUCLEO STM32F429ZI
GND	GND	CN8.13 / GND
VIN	+5V	CN8.9 / +5V
SCL	PB8	CN7.2 / D15
SDA	PB9	CN7.4 / D14

Valores programados: Utiliza el periférico I2C1, y durante la inicialización realiza una transferencia de 3 bytes para programar el registro de configuración del sensor VEML7700, con una ganancia de 0x18 y un tiempo de integración de 0x00. También configuramos un timer periódico de 1 segundo.

Cálculos realizados: Para leer la luminosidad del sensor utilizamos $luminosidad_raw = (buf_als[1] \ll 8) | buf_als[0]$ para colocar los bytes recibidos correctamente, y luego $luminosidad_lux = (float)(luminosidad_raw * 0.0601)$ para calcular la luminosidad entre 0 y 4000 lux, puesto que 0.0601 es 4000/66535 (la luminosidad máxima, entre la resolución máxima).

MÓDULO PRINCIPAL:

Valores programados: Utilizamos un timer virtual periódico de 250 ms, para cambiar el ciclo del LED RGB .

Cálculos realizados: Nos piden en modo REPOSO, cambiar el LED RGB del 10 al 90%, en 2 segundos, por lo que usamos $timer = \frac{2000ms}{8\text{ pasos}} = 250\text{ ms}$.

En el modo AUTO, calculamos la diferencia de luminosidad entre la referencia y la medida por el sensor, y modificamos el ciclo de trabajo del LED de alta luminosidad, en caso de que la diferencia sea mayor de 50 lux $if(abs(dif\ lum) > 50)$. Para almacenar las medidas en el buffer circular, y para leerlas, usamos los mismos cálculos que en el módulo COMPC. También resalta la estructura para actualizar las barras de luminosidad, que debe empaquetar y enviar I_sensor y $Iref$ (de 16 bits) en el array de 8 bits, realizando una fragmentación binaria.

En el modo PROGRAMACION, utilizamos un tick de parpadeo $tick = (osKernelGetTickCount() \% 500) < 250$, para alternar entre el espacio en blanco que genera el tick y el valor seleccionado para modificar. En la modificación de los valores, al incrementar o decrementar pulsando arriba o abajo respectivamente, usamos una sentencia que incrementa o decrementa el valor, siempre que no se haya alcanzado el límite, y si se alcanza, lo reinicia. Por ejemplo, con las horas, que al llegar a 23 pasan a 0 al incrementar, y de 0 pasan a 23 al decrementar.

SOFTWARE

2.3 Descripción de cada uno de los módulos del sistema

Módulo RGB:

Este módulo se encarga de encender y apagar los leds de la tarjeta de aplicación. Estos leds se pueden encender con un color determinado y un ciclo de trabajo seleccionado, por eso creamos una estructura con estos dos campos. Desde el inicio del hilo, activamos un timer periódico de 1ms y esperamos a recibir un mensaje de la cola. Cuando se recibe el mensaje, este indica el color y el porcentaje de ciclo de trabajo que queremos. En la callback del timer estaremos incrementando una cuenta, y si el ciclo de trabajo seleccionado es mayor que la cuenta, se pintará el LED del color seleccionado, si no, se mantendrán los pines de los 3 colores apagados.

Módulo JOYSTICK:

Módulo encargado de detectar e identificar una pulsación en el joystick. Tras recibir una pulsación, la callback de interrupciones externas salta y activamos el flag 0x01 para avisarlo. Desde el hilo del joystick, se espera a este flag y se dejan 25 ms de seguridad ante rebotes. A continuación, se identifica el gesto pulsado mediante HAL_GPIO_ReadPin. Mientras no se deje de pulsar, se arranca el timer periódico de 20ms y se entra en un bucle infinito que cuenta hasta 50 ticks ($20\text{ms} \times 50\text{ticks} = 1\text{s}$) para saber si la pulsación ha sido larga. Dentro del bucle, se espera a que la callback del timer active la flag 0x02, y se incrementa la cuenta al entrar. Además, antes de soltar, si la cuenta se hace mayor o igual a 50 ticks, se determina como pulsación larga y se espera a que suelte. Cuando se suelta el botón, si se ha contado hasta los 50 ticks, ya se determina que la pulsación es larga, y si no, corta.

Cuando se ha determinado la duración de la pulsación, se guarda junto al gesto pulsado en la estructura pulsación, que es enviada al hilo principal mediante la cola de mensajes del joystick.

Módulo HORA:

Es el encargado de la gestión del reloj del sistema. Se sincroniza con el resto del programa mediante 3 variables globales: horas, minutos y segundos. En el hilo del reloj, iniciamos un timer periódico de 1s, y en su callback incrementamos los segundos. Se ajustan los minutos y las horas, cuando segundos o minutos llegan a 59, y se ajustan las horas al llegar a 23. Todos reinician a 0 al llegar a su máximo. También usamos algunas funciones como void ModificarReloj(int h, int min, int seg), llamada desde el hilo principal o el de prueba, para el reloj, cambia la hora a la pedida (siempre que los valores sean correctos), y reinicia el timer. También hay una función void GetClock(int *h, int *m, int *s), para consultar la hora actual de forma segura.

Módulo POT:

Utilizando la plantilla entregada por los profesores conseguimos este código para el convertidor analógico-digital. En este caso deseamos que los cambios de tensión que se realizan en los potenciómetros se interpreten como valores digitales, para ello usamos la función ADC_getVoltage(ADC_HandleTypeDef *hadc, uint32_t Channel), que lee los valores del potenciómetro 1 en el canal 3, y del potenciómetro 2 en el canal 10. Esta función devuelve un float con el valor de los voltios entre 0.0 y 3.3, que convertimos en cada caso. Para el potenciómetro 1, queremos un valor entre 100 y 4000 lux, y para el potenciómetro 2, un porcentaje de ciclo de trabajo entre 5 y 95%. Después de convertir estos valores y asegurarnos que no se exceden los umbrales, actualizamos los valores enviándolos con la cola de mensajes, si los anteriores eran distintos.

Módulo LCD:

Es el encargado de representar información por el LCD. Este módulo configura todo lo relacionado con el uso del LCD: la inicialización de los pines, la función de reset del LCD (que configura la interfaz SPI), la función de inicialización del LCD (que escribe los comandos necesarios para su uso), una función para escribir comandos y otra para escribir datos, y una función para actualizar los datos que se muestran en el LCD. Cuenta con una función que permite borrar el buffer del LCD, para poder escribir nuevamente. También cuenta con las funciones que permiten escribir en la primera línea del LCD (que corresponde con las dos primeras páginas de este) o en la segunda línea (dos últimas páginas del LCD), además de una función que nos permite elegir en que línea queremos escribir, la cual utilizamos en el módulo principal y test. A parte de dibujar en estas líneas, cuenta con la funcionalidad de representar 2 barras que corresponden con la luminosidad de 2 variables. Este módulo extrae una estructura de la cola de mensajes, y dependiendo del valor de la línea, escribe en una línea u otra, o representa las barras de luminosidad.

Módulo COM-PC:

Encargado de recibir macros con comandos desde TeraTerm, procesarlos y responder. Este módulo utiliza las líneas Rx y Tx de USART3 para comunicarse con el PC, que está ya internamente conectado mediante el USB. El funcionamiento consiste en sacar de la USART, desde el hilo de recepción ThRX, los bytes enviados de Teraterm. Con cada byte se comprueba si la estructura es correcta (SOH CMD LEN Payload EOT), en este caso, limpiamos el mensaje para enviarlo a la cola de Rx. Para limpiar el mensaje usamos la función FRAME_PROCESS(), donde metemos el comando, la longitud y el payload en una estructura msg_RX. De esta cola sacan la estructura, el hilo principal o el de test, donde se procesará el comando y se realizarán las acciones pertinentes, para finalmente enviar la respuesta con el comando en CA2. Esta respuesta se envía a la cola de Tx mediante la función COM_EnviarRespuesta(), y se saca en el hilo de transmisión ThTX, donde FRAME_CREATOR() transforma el mensaje para que tenga la estructura correcta. Se le añade además `\r\n` para que desde Teraterm se vea un mensaje por línea. Finalmente, se devuelve el mensaje por la USART.

Módulo PWM:

Encargado de emitir un pulso de ciclo de trabajo ajustable, al recibirse por una cola de mensajes la orden. El código consiste en un hilo que saca mensajes de una cola mediante la función setIntensidad(), y, si el mensaje es el deseado, transforma el ciclo de trabajo al valor de anchura del pulso, y lo aplica mediante `__HAL_TIM_SET_COMPARE()`.

Módulo SENSOR LUMINOSIDAD:

Encargado de medir la luminosidad mediante el sensor VELM7700, conectado a la tarjeta NUCLEO mediante una protoboard. Inicialmente configura el sensor VEML7700 con sus 3 bytes de inicialización. A continuación, utiliza el bus I2C conectado al sensor para leer la temperatura, función que se realiza en el hilo, tras activarse el flag del timer de 1 segundo. Tras la espera, se accede a la función leerLuminosidad(), que transmite la dirección de lectura del registro de luz, y a continuación recibe la medida de 2 bytes con el dato crudo de iluminación. Este dato es desplazado y empaquetado para ser transformado matemáticamente al formato en lux entre 4000 y 100, y enviado por la cola de mensajes.

2.4 Descripción global del funcionamiento de la aplicación. Descripción del autómata con el comportamiento del software (si procede)

Comunicación entre módulos:

- Módulo RGB: utilizamos la función `encenderLeds(int porcentaje, int color)`, que pone el mensaje con los parámetros indicados, en la cola del RGB.
- Módulo JOYSTICK: sacamos el valor del gesto pulsado, directamente de la cola del joystick.
- Módulo HORA: utilizamos la función `ModificarReloj(int h, int min, int seg)`, que pone actualiza las variables globales de horas, minutos y segundos.
- Módulo POT: sacamos los valores de los potenciómetros, directamente de la estructura que sale de la cola de potenciómetros.
- Módulo LCD: metemos a la cola del LCD, la estructura que contiene la longitud, el mensaje a enviar, y la línea en la que escribir.
- Módulo COM PC: sacamos de la cola de recepción, la estructura que contiene el comando, el payload y su longitud, y enviamos la respuesta con la función `COM_EnviarRespuesta()`, que empaqueta de nuevo el mensaje desde el módulo principal, y lo envía a la cola de transmisión.
- Módulo PWM: utilizamos la función `setIntensidad(uint8_t porcentaje_ciclo)`, que pone el mensaje con el parámetro indicados, en la cola del PWM.
- Módulo SENSOR LUMINOSIDAD: sacamos el valor de luminosidad medida por el sensor, directamente de la cola del sensor.

Inicialización: ponemos la hora a 09:50:00, la luminosidad de referencia a 1500 lux, la intensidad del LED de alta luminosidad a 0, los LEDs RGB apagados y el modo a REPOSO.

El autómata principal se repite cada 100ms, con los siguientes estados:

MODO REPOSO: escribimos el mensaje “SBM 2026” en la línea 1, y la hora en la línea 2.

A su vez, en la primera vuelta al entrar al modo, iniciamos el timer virtual de 250 ms, en cuya callback aumentamos el ciclo de trabajo de 10 a 90, en saltos de 10, y luego lo bajamos de 90 a 10. De esta forma, el timer se inicia en la primera vuelta cada vez que entramos a modo reposo, y se encenderá el LED RGB de color verde, aumentando de 10-90% en 2 segundos, y luego disminuyendo en el mismo tiempo, periódicamente.

Por último, si se recibe una pulsación larga del centro del joystick, se pasará a modo manual, parando el timer y poniendo el valor del joystick a 0 para que no capte continuamente la pulsación.

MODO MANUAL: escribimos el mensaje “M-PWM” seguido del porcentaje de ciclo de trabajo del LED de alta luminosidad, en la línea 1, y la hora en la línea 2.

Por otro lado, sacamos la luminosidad del sensor de la cola (entregada cada segundo), y los valores de los potenciómetros. Con el valor del potenciómetro 2 asignamos la intensidad del LED de alta luminosidad. Usamos los valores del potenciómetro 1 y de la luminosidad del sensor, para dibujar las barras en el LCD, y para encender el LED RGB de color rojo, cuando la diferencia de lúmenes entre ambos valores sea mayor de 250.

Por último, si se recibe una pulsación larga del centro del joystick, se pasará a modo auto, apagando los LEDs y poniendo el valor del joystick a 0 para que no capte continuamente la pulsación.

MODO AUTO: escribimos el mensaje “A-PWM” seguido del porcentaje de ciclo de trabajo del LED de alta luminosidad, en la línea 1, y la hora en la línea 2.

Por otro lado, sacamos la luminosidad del sensor de la cola (entregada cada segundo), y la comparamos con la luminosidad de referencia general (no la del potenciómetro 1 fijada en el modo anterior). Si la diferencia entre ambas luminosidades es mayor de 50, ajustamos el ciclo de trabajo del LED de alta luminosidad, para lograr la luminosidad de referencia de forma automática.

Usamos los valores de luminosidad de referencia general y de luminosidad del sensor, para dibujar las barras en el LCD, y para encender el LED RGB de color azul, cuando la diferencia de lúmenes entre ambos valores sea mayor de 250.

A su vez, cada segundo, guardaremos las medidas de hora, luminosidad del sensor y de referencia y ciclo de trabajo del LED de alta luminosidad, en un buffer circular de 20 medidas, que se reescribirá al llegar al final.

Por último, si se recibe una pulsación larga del centro del joystick, se pasará a modo programación, apagando los LEDs y poniendo el valor del joystick a 0 para que no capte continuamente la pulsación.

MODO PROGRAMACIÓN: escribimos el mensaje “PROGRAMACION” en la línea 1, y la hora, junto con la luminosidad de referencia en la línea 2.

En la función modoProgramacionHora(), atendemos a las pulsaciones del joystick que permiten cambiar las medidas de hora y luminosidad de referencia. Inicialmente establece el campo de horas para ser modificado y extrae los valores actuales de las horas, minutos y segundos, así como la luminosidad de referencia. A continuación:

- Si recibe una pulsación central corta, modifica los valores.
- Si recibe una pulsación hacia arriba, aumenta los campos hasta su límite, y los reinicia en la siguiente pulsación. Las horas pasan de 23 a 00, los minutos y segundos de 59 a 00, y lref de 4000 a 100.
- Si recibe una pulsación hacia abajo, disminuye los campos hasta su límite, y los reinicia en la siguiente pulsación. Las horas pasan de 00 a 23, los minutos y segundos de 00 a 59, y lref de 100 a 4000.
- Si recibe una pulsación hacia la derecha, cambia el campo modificado (horas -> minutos -> segundos -> lref), y reinicia a horas.
- Si recibe una pulsación hacia la izquierda, cambia el campo modificado (lref -> segundos -> minutos -> horas), y reinicia a lref.

Finalmente, utiliza el tick de parpadeo para intercalar espacios y la medida seleccionada al representarla en el LCD.

En la función comandosTeraterm(), recibe la estructura del mensaje, de la cola de recepción, y realiza una acción u otra, según el comando.

- Con CMD_PUESTA_HORA, comprueba la validez de la hora recibida y responde usando la función COM_EnviarRespuesta() con el comando en Ca2.
- Con CMD_ESTABLECE_LREF, comprueba la validez de la luminosidad recibida y responde usando la función COM_EnviarRespuesta() con el comando en Ca2.
- Con CMD_GET_MEDIDAS, responde usando la función COM_EnviarRespuesta() con el comando en Ca2, y el buffer de medidas con aquellas medidas correspondientes a una medición. Repite este proceso hasta que no quedan medidas por leer en el buffer. Entre medidas, espera 25 ms.
- Con CMD_DEL_MEDIDAS, responde usando la función COM_EnviarRespuesta() con el comando en Ca2, y borra con memset() el buffer de medidas, a la vez que reestablece los índices.

Por último, si se recibe una pulsación larga del centro del joystick, se pasará a modo programación, apagando los LEDs y poniendo el valor del joystick a 0 para que no capte continuamente la pulsación.

2.5 Descripción de las rutinas más significativas que ha implementado.

1. ThPrincipal

Es el bucle central del sistema, que implementa los cuatro estados, coordina la lectura de las colas y el funcionamiento real del proyecto.

2. modoProgramacionHora

Es la función encargada de gestionar la modificación de las medidas mediante la recepción de gestos del joystick, asegurando que cumplen los límites establecidos.

3. comandosTeraterm

Función encargada de implementar las acciones necesarias al recibir los distintos comandos de Teraterm, así como de enviar una respuesta.

4. leer_Luminosidad

Función que establece la comunicación con el sensor VEML7700, y a continuación realiza la medición de luminosidad, obteniendo 2 bytes a convertir en el formato deseado. Tras una comprobación de que se respetan los límites, añade el valor a la cola del sensor.

5. LCD_DibujarBarrasLuminosidad

Esta función dibuja las barras de luminosidad usando las columnas 110 y 120. Realiza un bucle inverso desde la página 3 a la página 0, de forma que, si quedan 8 o más píxeles por pintar, se rellena la página actual completa con 0xFF, si quedan entre 0 y 8 píxeles por pintar, se rellenan los píxeles superiores necesarios, y si quedan 0 píxeles por pintar, se borra con 0x00.

6. symbolToLocalBuffer

Esta función permite dibujar fácilmente en el LCD el mensaje deseado, en la línea 1 o 2, según se indique. Llama a las funciones LCD_symbolToLocalBuffer_L1 y LCD_symbolToLocalBuffer_L2, respectivamente.

7. COM_EnviarRespuesta

Esta función empaqueta el mensaje de respuesta para enviarlo a la cola de transmisión, limpiando previamente la estructura, y copiando el payload cuando tiene contenido, evitando desbordar el buffer.

8. FRAME_PROCESS

Función que procesa la trama recibida de Teraterm y la empaqueta en una estructura para que sea leída fácilmente por la cola de recepción.

9. FRAME_CREATOR

Función que procesa la trama para responder a Teraterm y la empaqueta con la estructura adecuada, incluyendo \r\n para una mejor visualización.

10. ADC_getVoltage

Función de los potenciómetros, que devuelve un float con el valor de los voltios, entre 0.0 y 3.3, correspondientes al potenciómetro indicado mediante el canal (POT1->canal 3, POT2-> canal 10), que convertimos en cada caso.

3 DEPURACION Y TEST

3.1 Pruebas realizadas.

Test RGB: consiste en modificar el color y el ciclo de trabajo de los LEDs, de forma periódica, y enviarlos a la cola con la función `encenderLeds()`.

1. El objetivo de este test es verificar el funcionamiento correcto del LED RGB, asegurando el correcto control del ciclo de trabajo y el color adecuado. Se utilizan los módulos `rgb.c` y `rgb.h`, aunque más adelante se utiliza en el módulo principal.
2. El proyecto se encuentra en el módulo RGB, llamado `Proy_RGB`.
3. Condiciones de entrada:
 - a. Tarjeta NUCLEO-STM32F429ZI conectada a la mbed-app-board.
 - b. Entorno Keil en modo depuración.
 - c. Ejecución del programa iniciada.
4. Resultados esperados: Inicialmente se enciende el LED de color rojo con un ciclo de trabajo del 10%, y se aumenta un 10% cada segundo, hasta llegar al 90%. A continuación, se enciende el LED con color verde, al 50% de su ciclo de trabajo, durante 5 segundos. Finalmente, se enciende el LED con color azul, al 90% de ciclo de trabajo, durante 5 segundos. Este funcionamiento se repite periódicamente.
Resultados obtenidos: los colores concuerdan con los esperados. Se puede ver el cambio de luminosidad con los distintos ciclos de trabajo, de forma que la prueba ha resultado correcta.

Test JOYSTICK: consiste en esperar continuamente a sacar los valores de la cola del joystick.

1. El objetivo de este test es verificar el funcionamiento correcto de los gestos UP, DOWN, CENTER, RIGHT y LEFT, asegurando lecturas correctas de pulsaciones largas y cortas. Se utilizan los módulos `Joystick.c` y `Joystick.h`, aunque más adelante se utiliza en el módulo principal.
2. El proyecto se encuentra en el módulo JOYSTICK, llamado `Proy_Joystick`.
3. Condiciones de entrada:
 - a. Tarjeta NUCLEO-STM32F429ZI conectada a la mbed-app-board.
 - b. Entorno Keil en modo depuración, con un watch en la variable leído.
 - c. Ejecución del programa iniciada.
4. Resultados esperados:
 - a. Al pulsar los distintos gestos, se detecta el gesto adecuado en `leido.tipo` (arriba-> 1, derecha-> 2, abajo-> 3, izquierda-> 4, centro-> 5).
 - b. Al realizar una pulsación de 1 segundo o mayor, se detecta en `leido.larga` (`larga->1`, `corta->0`), de forma que en cuanto la pulsación ha superado el segundo, se establece como larga, aunque no se suelte.
 - c. Al realizar varias pulsaciones seguidas rápido, se detectan todas correctamente.

Resultados obtenidos: los valores obtenidos muestran de forma adecuada los gestos. No se identifica la duración de la pulsación hasta que no se suelta, y el valor del gesto y duración llegan a la vez, sin mezclarse con valores anteriores. Al realizar varias pulsaciones seguidas, no se aprecian rebotes, captando todos los gestos correctamente. Por tanto, se considera que la prueba es correcta.

Test HORA: consiste en la espera inicial de 5 segundos (durante los cuales el reloj contará desde 00:00:00 a 00:00:05), y a continuación, la modificación de la hora a las 12:59:25 mediante el uso de la función `ModificarReloj()`, que envía cambia las variables globales.

1. El objetivo de este test es verificar el funcionamiento correcto del timer del reloj y asegurar la correcta modificación de la hora. Se utilizan los módulos `clock.c` y `clock.h`, aunque más adelante se utiliza en el módulo principal.
2. El proyecto se encuentra en el módulo HORA, llamado `Proy_Hora`.
3. Condiciones de entrada:
 - a. Entorno Keil en modo depuración, con un watch en las variables horas, minutos y segundos.
 - b. Ejecución del programa iniciada.
4. Resultados esperados:
 - a. La hora inicia en 00:00:00.
 - b. Los segundos se modifican correctamente cada segundo.
 - c. Los segundos y minutos pasan de 59 a 00, y las horas de 23 a 00.
 - d. Al modificar la hora, esta se implementa correctamente.

Resultados obtenidos: la hora inicia correctamente, y los cambios se realizan correctamente. Este test no prueba el paso de 23h a 00h. Al modificar la hora, esta cambia correctamente. El test es correcto.

Test POT: consiste en esperar continuamente a sacar los valores de la cola del potenciómetro.

1. El objetivo de este test es verificar el funcionamiento correcto de los potenciómetros POT1 y POT2, asegurando lecturas correctas al ajustarlos, y precisión en los valores mínimos y máximos. Se utilizan los módulos `pot.c` y `pot.h`, aunque más adelante se utiliza en el módulo principal.
2. El proyecto se encuentra en el módulo POT, llamado `Proy_POT`.
3. Condiciones de entrada:
 - a. Tarjeta NUCLEO-STM32F429ZI conectada a la mbed-app-board.
 - b. Entorno Keil en modo depuración, con un watch en la variable `valorLeido_test`.
 - c. Ejecución del programa iniciada.
4. Resultados esperados:
 - a. Al rotar el POT1 a sus extremos, debemos obtener 100 lux y 4000 lux respectivamente, en `valorLeidoTest.lux_pot1`, y los valores intermedios deben variar de forma fluida y lineal.
 - b. Al rotar el POT2 a sus extremos, debemos obtener 5% y 95% respectivamente, en `valorLeidoTest.ciclo_pot2`, y los valores intermedios deben variar de forma fluida y lineal.

Resultados obtenidos: los valores capturados varían con soltura y exactitud, alcanzando los límites correctamente. La prueba es correcta.

Test LCD: consiste en meter en la cola del LCD, la estructura con la línea donde se desea escribir (líneas 1 y 2), el contenido que se desea escribir y la longitud de este. No hay test para las barras.

1. El objetivo de este test es verificar el funcionamiento correcto de la escritura en el LCD. Se utilizan los módulos `lcd.c` y `lcd.h`, aunque más adelante se utiliza en el módulo principal.
2. El proyecto se encuentra en el módulo LCD, llamado `Proy_Lcd`.
3. Condiciones de entrada:
 - a. Tarjeta NUCLEO-STM32F429ZI conectada a la mbed-app-board.
 - b. Entorno Keil en modo depuración.
 - c. Ejecución del programa iniciada.

4. Resultados esperados:

- a. Al escribir en la línea 1 o 2, debe verse en el LCD la información escrita correctamente, sin parpadeos notables.
- b. Al reescribir la información, el buffer se debe haber borrado correctamente, sin dejar rastros de la escritura anterior.

Resultados obtenidos: los valores visualizados son correctos. El LCD se borra y reescribe limpiamente, y no se perciben parpadeos. La prueba se estima correcta, a falta de un test de las barras (aunque en el módulo principal funciona correctamente).

Test COM_PC: consiste en sacar de la cola RX la estructura con la trama enviada por TeraTerm, limpia, y realizar las distintas acciones (cambiar la hora, la luminosidad, mandar las medidas o borrarlas) según el comando recibido.

1. El objetivo de este test es verificar el funcionamiento correcto de la recepción de comandos mediante TeraTerm, la realización de las acciones pertinentes, la correcta respuesta a estos y comprobar que se ignoran los mensajes incorrectos. Se utilizan los módulos com.c y com.h, aunque más adelante se utiliza en el módulo principal.
2. El proyecto se encuentra en el módulo COM_PC, llamado Proy_Com_PC.
3. Condiciones de entrada:
 - a. Cable USB conectado a la tarjeta NUCLEO-STM32F429ZI.
 - b. Entorno Keil en modo depuración, con la ejecución del programa iniciada
 - c. TeraTerm abierto y configurado para la comunicación con la tarjeta.
 - d. Macros con los distintos comandos preparadas en la carpeta COM_PC.
4. Resultados esperados:
 - a. Al recibir el comando de puesta en hora, se cambian las horas a las indicadas y se responde correctamente con el comando en ca2.
 - b. Al recibir el comando de establecer lref, se cambia la luminosidad de referencia a la indicada y se responde correctamente con el comando en ca2.
 - c. Al recibir el comando de get medidas, se envían todas las medidas distintas de 0 que haya en el buffer circular, mostrándose correctamente en TeraTerm, con el comando en ca2.
 - d. Al recibir el comando de delete medidas, se borra el contenido del buffer circular y se responde correctamente con el comando en ca2.
 - e. Al volver a pedir las medidas tras borrarlas, no se recibe respuesta, ya que no hay contenido.

Resultados obtenidos: los comandos realizan correctamente sus respectivas acciones, y en TeraTerm se ven los bytes de estructura como figuras extrañas, al ser ASCII. Prueba correcta.

Test PWM: consiste en modificar el ciclo de trabajo del LED, de forma periódica. Para ello, enviamos a la cola, con la función setIntensidad(), el valor del ciclo de trabajo que queremos. Incrementamos el ciclo desde 0 a 100% y viceversa, cada 200ms.

1. El objetivo de este test es verificar el funcionamiento correcto del LED de alta luminosidad, asegurando luminosidad máxima y mínima, y el correcto control del ciclo de trabajo. Se utilizan los módulos pwm.c y pwm.h, aunque más adelante se utiliza en el módulo principal.
2. El proyecto se encuentra en el módulo PWM, llamado Proy_Pwm.
3. Condiciones de entrada:
 - a. LED de alta luminosidad correctamente conectado y visible.
 - b. Entorno Keil en modo depuración.
 - c. Ejecución del programa iniciada

4. Resultados esperados: La luminosidad del LED parte de 0 y se incrementa y decrementa periódicamente, llegando a apagarse y a encenderse del todo. La frecuencia está configurada para entregar una luminosidad alta.

Resultados obtenidos: los cambios en la luminosidad son notables y correctos, el LED se llega a apagar, y en su máxima iluminación tiene alta luminosidad. El test se considera correcto.

Test SENSOR LUMINOSIDAD: consiste en esperar continuamente a sacar los valores de la cola del sensor de luminosidad.

1. El objetivo de este test es verificar el funcionamiento correcto del sensor, asegurando las lecturas entre 0 y 4000 lux, coherentes con los cambios de luminosidad. Se utilizan los módulos luminosidad.c y luminosidad.h, aunque más adelante se utiliza en el módulo principal.
2. El proyecto se encuentra en el módulo SENSOR_LUMINOSIDAD, llamado Proy_sensor_lumi.
3. Condiciones de entrada:
 - a. Tarjeta NUCLEO-STM32F429ZI conectada mediante protoboard al sensor de luminosidad.
 - b. Entorno Keil en modo depuración, con un watch en la variable l_test.
 - c. Ejecución del programa iniciada
4. Resultados esperados: lecturas cercanas a 0 lux, en situaciones de poca luminosidad, y lecturas cercanas a 4000 lux, al situar una linterna cerca del sensor.
Resultados obtenidos: los valores capturados varían con soltura y exactitud, alcanzando los límites correctamente. La prueba es correcta.